Lab Project 2 File Transfer over TCP

Group size: One or two students

Language: C on Linux

Objective

Create a TCP client and server that transfer one file from the client to the server. The programs must preserve every byte so that text and binary files can be copied correctly.

Program Interfaces

Start the server first with one argument, the listening port:

```
./server <port>
```

Run the client with the server address, server port, source file, and destination filename:

```
./client <server_ip> <server_port> <source_file> <destination_file>
```

Example when both programs run on the same computer:

```
./server 9000
./client 127.0.0.1 9000 file1 file2
```

Transfer Protocol

1. The server creates a TCP socket, binds it to the selected port, and listens for clients.
2. The client opens the source file, creates a TCP socket, and connects to the server.
3. The client sends the destination filename, including its terminating null byte.
4. The server receives the complete filename and opens the destination file in binary write mode.
5. The server sends the confirmation string received! to the client.
6. The client reads and sends each file block, while periodically reporting bytes sent and throughput.
7. The server writes each received block and periodically reports bytes received and throughput.
8. The client calls shutdown() to mark the end of the byte stream. The server then closes the client connection and waits for the next client.

The destination filename may contain at most `FILENAME_SIZE - 1` bytes. With the supplied code, this limit is 1023 bytes. Opening the destination with `fopen(filename, "wb")` creates the file or replaces the contents of an existing file.

Deliverables

- **Demonstrate the project to the TA using two terminal windows or two computers.**
- **Submit client.c and server.c to Camino.**

Client and Server Flow

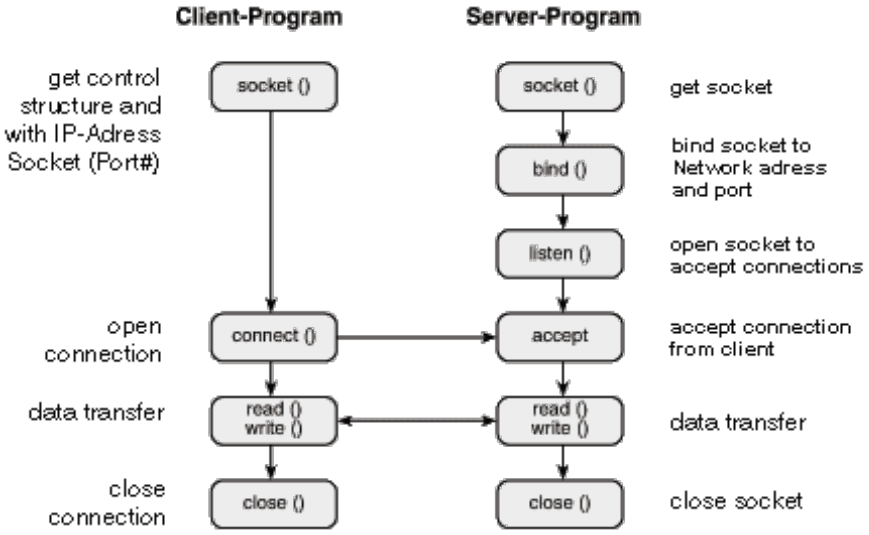


Figure 1 Basic socket operations used by the client and server

Socket Creation

Both programs begin by creating an IPv4 TCP socket. socket() returns a file descriptor that identifies the socket in later calls.

```
int socket(int domain, int type, int protocol);
```

- domain is AF_INET for IPv4.
- type is SOCK_STREAM for a reliable, connection-oriented byte stream.
- protocol is 0, which asks the operating system to select the standard protocol for AF_INET and SOCK_STREAM: TCP.

The server stores its listening descriptor in socket_fd:

```
int socket_fd = socket(AF_INET, SOCK_STREAM, 0);
```

The client stores its connected socket descriptor in sockfd:

```
int sockfd = socket(AF_INET, SOCK_STREAM, 0);
```

Server Address and Bind

The server clears its IPv4 address structure before assigning its address family, local address, and port number.

```
struct sockaddr_in serv_addr;
memset(&serv_addr, 0, sizeof(serv_addr));

serv_addr.sin_family = AF_INET;
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
serv_addr.sin_port = htons(port);
```

INADDR_ANY represents 0.0.0.0. It allows the server to accept connections addressed to any local IPv4 interface, including the loopback address and an assigned Ethernet or Wi-Fi address.

To restrict the server to connections from the same computer, use htonl(INADDR_LOOPBACK), which represents 127.0.0.1.

bind() associates the listening socket with this local address and port:

```
if (bind(socket_fd,
         (struct sockaddr *)&serv_addr,
         sizeof(serv_addr)) < 0) {
    perror("Could not bind socket");
    close(socket_fd);
    return EXIT_FAILURE;
}
```

Listen and Backlog

listen() marks the bound socket as a listening socket. BACKLOG is 10 in the supplied server.

```
if (listen(socket_fd, BACKLOG) < 0) {
    ...
}
```

The backlog is the requested maximum number of established connections that may wait for accept(). It does not create concurrent client handlers. This server processes one client at a time, then returns to accept() inside its while loop.

Accepting a Client

accept() removes one waiting connection from the queue and returns a new descriptor for communication with that client. The listening descriptor remains available for later clients.

```
int connection_fd =
    accept(socket_fd,
           (struct sockaddr *)&client_addr,
           &client_length);
```

- socket_fd listens for new connection requests.
- connection_fd transfers the filename and file data for the accepted client.

Connecting the Client

The client converts the server address from text to binary with inet_pton(), then connects its socket to that address and port.

```
serv_addr.sin_family = AF_INET;
serv_addr.sin_port = htons(port);

if (inet_pton(AF_INET, argv[1], &serv_addr.sin_addr) != 1) {
    ...
}

if (connect(sockfd,
            (struct sockaddr *)&serv_addr,
            sizeof(serv_addr)) < 0) {
    ...
}
```

Use 127.0.0.1 when the client and server run on the same computer. When they run on different computers, use an IPv4 address assigned to the server. A client should not use 0.0.0.0 as the destination address.

`inet_pton()` converts an IP address written as text into the binary format required by the Socket API.

Its name means:

```
inet = Internet address
p    = presentation format, such as "127.0.0.1"
n    = network format, the binary representation
```

Destination Filename Handshake

The client sends the destination filename before sending file data. The added 1 includes the terminating null byte, which lets the server identify the end of the filename.

```
size_t filename_bytes = strlen(argv[4]) + 1;

write_all(sockfd, argv[4], filename_bytes);
```

The server uses read_filename() to read until it receives that null byte. It rejects an empty filename or a filename that does not fit in the destination buffer.

```
if (read_filename(connection_fd,
                  filename,
                  sizeof(filename)) < 0) {
    ...
}
```

After successfully opening the destination file, the server sends the confirmation string received!. ACK_SIZE is 10 because sizeof() counts the nine visible characters and the terminating null byte.

```
#define ACK_MESSAGE "received!"
#define ACK_SIZE (sizeof(ACK_MESSAGE))

write_all(connection_fd, ACK_MESSAGE, ACK_SIZE);
```

The client uses read_all() because one read() call is not guaranteed to return all 10 acknowledgement bytes.

Sending File Data

The client reads the source file in blocks. fread() returns the number of bytes placed in file_buff. write_all() sends that entire block, even if one write() call sends only part of it.

```
size_t bytes_read =
    fread(file_buff, 1, sizeof(file_buff), source_file);

if (ferror(source_file)) {
    ...
}

if (write_all(sockfd, file_buff, bytes_read) < 0) {
    ...
}
```

BUFFER_SIZE is 4096 bytes. It controls the application buffer size; it does not define a TCP segment or preserve a message boundary.

Receiving File Data

The server repeatedly reads from connection_fd. A positive return value is the number of bytes received during that call.

```
ssize_t net_bytes_read =
    read(connection_fd, net_buff, sizeof(net_buff));

if (net_bytes_read > 0)
    fwrite(net_buff, 1, (size_t)net_bytes_read, dest_file);
```

- A positive value means that many bytes were received.
- Zero means the client has finished sending and the byte stream has reached end-of-file.
- A negative value means an error occurred; EINTR may be retried.

The server writes exactly net_bytes_read bytes. The data may be binary, so net_buff is not treated as a C string and does not need bzero(), memset(), or a terminating null byte.

Reporting Transfer Progress

Both programs print transfer statistics approximately once per second, but they use different timing APIs and output formats. The client uses a monotonic clock; the server uses gettimeofday(). In both programs, the timing check occurs after a file or socket operation returns.

Client Progress Report

The client enables POSIX clock declarations, defines a one-second reporting interval, and disables standard-output buffering so progress is visible immediately even when output is redirected:

```
#define _POSIX_C_SOURCE 200809L
#include <time.h>
#define REPORT_INTERVAL_SECONDS 1.0

setvbuf(stdout, NULL, _IONBF, 0);
```

CLOCK_MONOTONIC measures elapsed time and is not affected if the system date or wall clock changes. The client counts both the bytes in the current reporting interval and the total bytes sent:

```
unsigned long long total_sent = 0;
unsigned long long interval_sent = 0;
struct timespec interval_start;

clock_gettime(CLOCK_MONOTONIC, &interval_start);
...
total_sent += bytes_read;
interval_sent += bytes_read;
```

After obtaining the current monotonic time, the client calculates the elapsed seconds and reports when at least one second has passed. It also reports remaining interval bytes when end-of-file is reached:

```
double elapsed = (double)(now.tv_sec - interval_start.tv_sec) +
    (double)(now.tv_nsec - interval_start.tv_nsec) / 1e9;

if (elapsed >= REPORT_INTERVAL_SECONDS ||
    (bytes_read == 0 && interval_sent > 0)) {
...
}
```

Server Progress Report

The server uses gettimeofday(), which returns wall-clock time with microsecond fields. It tracks the cumulative total and remembers the total from the preceding report:

```
#include <sys/time.h>

unsigned long long total_received = 0;
unsigned long long last_report_received = 0;
struct timeval last_report = {0, 0};
```

After writing a block to the file, the server calculates the bytes and time in the current reporting window. Its displayed byte count is the cumulative total:

```
total_received += (unsigned long long)net_bytes_read;
...
unsigned long long bytes_this_window =
    total_received - last_report_received;
double seconds = (double)elapsed_us / 1000000.0;
double throughput_mbps =...

printf("Received %llu bytes so far; "
       "throughput = %.2f Mbps\n",
       total_received, throughput_mbps);
```

Throughput is calculated as interval bytes times 8 bits per byte, divided by elapsed seconds and then by 1,000,000 to obtain megabits per second. The client line shows interval bytes and a separate cumulative Total; the server line shows the cumulative received bytes. A short transfer may finish before an intermediate report is due, but both programs print a final cumulative byte total.

The one-second interval is approximate. If read(), write_all(), fread(), or fwrite() blocks, no progress line is printed until that operation returns and the timing check runs.

Partial Reads and Writes

TCP is a byte stream. It does not preserve the boundaries between individual write() calls. read() may therefore return fewer bytes than the sender wrote in one call, and write() may send fewer bytes than

requested. The supplied read_all() and write_all() helpers repeat these operations until the required number of bytes has been processed or an error or connection closure occurs.

Ending the Transfer

After sending the final file block, the client closes only the writing side of its socket:

```
shutdown(sockfd, SHUT_WR);
```

This tells TCP that the client has finished sending while leaving the client able to read. TCP first delivers any file data still waiting in the client's send buffer and then sends an end-of-stream notification (a TCP FIN).

The server's read() returns 0 only after all previously sent bytes have been read and the FIN has arrived. If unread file data remains, read() returns a positive byte count first; a later read() returns 0. The server then closes the destination file and connection_fd. When the client detects that closure, it closes sockfd and exits.

There is no fixed waiting time. On the same computer this is normally nearly immediate, and on a local network it is usually a few milliseconds. A remote transfer depends on latency, congestion, and how much data remains buffered. Because the supplied program uses a blocking read() without a timeout, it may wait much longer if the peer or network becomes unresponsive.

IPv4 Address Structure

The sockaddr_in structure stores an IPv4 address and a 16-bit TCP or UDP port number.

```
struct sockaddr_in {
    sa_family_t    sin_family;
    in_port_t      sin_port;
    struct in_addr sin_addr;
};

struct in_addr {
    uint32_t s_addr;
};
```

- sin_family is AF_INET for IPv4.
- sin_port stores the 16-bit port in network byte order.
- sin_addr.s_addr stores the 32-bit IPv4 address in network byte order.

Network Byte Order

A multi-byte number can be arranged in big-endian or little-endian order. Internet protocols use big endian as network byte order so that computers with different processor architectures interpret numeric network fields consistently.

```
Value:          0x12345678
Big endian:     12 34 56 78
Little endian:  78 56 34 12
```

htons() converts a 16-bit value from host byte order to network byte order.
htonl() converts a 32-bit value from host byte order to network byte order.
ntohs() converts a 16-bit value from network byte order to host byte order.

```
serv_addr.sin_port = htons(port);
serv_addr.sin_addr.s_addr = htonl(INADDR_ANY);
unsigned int client_port = ntohs(client_addr.sin_port);
```

INADDR_ANY has the numeric value zero, so changing its byte order does not change its bytes. htonl() is still used to show that s_addr expects a 32-bit value in network byte order.

Build and Test

Compile both programs on Linux:

```
gcc -std=c11 -Wall -Wextra -Wpedantic client.c -o client
gcc -std=c11 -Wall -Wextra -Wpedantic server.c -o server
```

Start the server in one terminal:

```
./server 9000
```

Run the client in another terminal:

```
./client 127.0.0.1 9000 file1 file2
```

Verify that the copied file is identical to the source:

```
cmp file1 file2
```

cmp produces no output when the files are identical. Test the program with both a text file and a binary file.

Throughput Experiment

In this experiment, create files of several sizes, transfer each file, and measure how file size affects TCP throughput. The client and server report transferred bytes and throughput while a transfer is in progress.

Create Test Files with dd

The dd command copies a selected number of fixed-size blocks from an input source to an output file. Run these commands in the directory that contains the client program:

```
dd if=/dev/zero of=file_1MiB.bin bs=1M count=1
dd if=/dev/zero of=file_10MiB.bin bs=1M count=10
dd if=/dev/zero of=file_100MiB.bin bs=1M count=100
```

- **if=/dev/zero** selects /dev/zero as the input. It supplies zero-valued bytes as needed.
- **of=filename** specifies the output file that dd creates.
- **bs=1M** sets the block size to 1 MiB, or 1,048,576 bytes, on GNU/Linux.
- **count=N** copies N blocks. Therefore, file size = block size × block count.

Caution: dd replaces an existing output file without asking for confirmation. Check the name after of= before running the command. Use ls -lh file_*MiB.bin to confirm the generated file sizes.

Optional larger file: If the 100 MiB transfer finishes too quickly to show several one-second reports, create a 500 MiB file with the following command, provided that enough storage is available.

```
dd if=/dev/zero of=file_500MiB.bin bs=1M count=500
```

Measure Throughput

1. Start the server in one terminal: ./server 9000
2. Transfer one test file from a second terminal. Use /usr/bin/time -p to record the elapsed wall-clock time.

```
/usr/bin/time -p ./client 127.0.0.1 9000 file_1MiB.bin received_1MiB.bin
```

3. Record the throughput values printed by the client and the server. Both programs report at approximately one-second intervals. The client also reports its final partial interval, but the server may show no throughput line when a transfer finishes in less than one second. Both programs still print the final byte total.
4. Verify the received file before continuing:

```
cmp file_1MiB.bin received_1MiB.bin
```

5. Repeat the transfer three times for each file size. Use the same network path and avoid other large transfers during the experiment. If the client and server are on different computers, replace 127.0.0.1 with the server computer's IPv4 address.